

LAPORAN PROGRAM
APLIKASI SEARCHING & SORTING
(C# WINDOWS FORM)

Disusun Oleh:

Nama : [MAHMUDI DHANI PRATAMA]

[FARHAN DWI RAMADHAN]

NIM : [2511004]

[2511008]

Mata Kuliah: Pemograman Lanjut

ANALISIS HASIL SEARCHING DAN SORTING

A. Analisis Searching

1. Perbandingan Sequential vs Binary Search

Pengujian dilakukan terhadap tiga data: nilai **78 (awal)**, **68 (tengah)**, dan **73 (akhir)**.

Nilai	Posisi Data	Sequential Search (Langkah)	Binary Search (Langkah)
78	Awal	1 Langkah	4 langkah
68	Tengah	±21 Langkah	3 langkah
73	Akhir	±25 Langkah	5 langkah

Analisis:

- Sequential Search lebih cepat jika data berada di awal.
- Binary Search lebih stabil dan efisien karena membagi data menjadi dua setiap langkah.
- Semakin besar data, Binary Search jauh lebih unggul.

2. Kasus Nilai Duplikat

Pengujian dilakukan pada nilai **55** dan **91**.

Hasil:

- Sequential Search berhasil menemukan semua data duplikat karena memeriksa seluruh elemen satu per satu.
- Binary Search hanya menemukan salah satu posisi dari data duplikat.

Penjelasan:

Binary Search bekerja dengan membagi data terurut menjadi dua bagian, sehingga ketika menemukan nilai yang dicari, proses berhenti tanpa menjamin menemukan semua data yang sama.

3. Kesimpulan Searching

- **Sequential Search:**
 - Cocok untuk data kecil atau tidak terurut
 - Lebih sederhana
- **Binary Search:**
 - Lebih efisien untuk data besar
 - Wajib menggunakan data yang sudah terurut

Secara umum, **Binary Search lebih efisien**, tetapi Sequential lebih fleksibel.

B. Analisis Sorting

1. Tabel Perbandingan Hasil

Algoritma	Perbandingan	Pertukaran
Bubble Sort	Tinggi	Tinggi
Selection Sort	Tinggi	Rendah
Insertion Sort	Sedang	Sedang
Counting Sort	0	0

2. Analisis Tiap Algoritma

- **Bubble Sort**
Menukar elemen berulang-ulang meskipun tidak diperlukan, sehingga banyak terjadi pertukaran yang tidak efisien.
- **Selection Sort**
Selalu mencari nilai minimum, sehingga jumlah pertukaran lebih sedikit, tetapi tetap banyak melakukan perbandingan.
- **Insertion Sort**
Memindahkan elemen ke posisi yang tepat, lebih efisien untuk data kecil dan hampir terurut.
- **Counting Sort**
Tidak menggunakan perbandingan, langsung menghitung frekuensi nilai, sehingga sangat cepat.

3. Pengaruh Data Tidak Terurut

Karena data awal acak:

- Bubble Sort dan Selection Sort melakukan banyak langkah tambahan
- Insertion Sort cukup adaptif terhadap kondisi data
- Counting Sort tetap stabil dan tidak terpengaruh

Algoritma yang paling banyak langkah ekstra: **Bubble Sort**

4. Kesimpulan Sorting

- Paling sedikit perbandingan: **Counting Sort**
- Paling sedikit pertukaran: **Selection Sort**
- Paling efisien secara keseluruhan: **Counting Sort**
- Paling cocok untuk data nilai mahasiswa:
Insertion Sort (karena sederhana, stabil, dan cukup cepat untuk data kecil-menengah)

C. Kesimpulan Umum

Kesulitan yang Dihadapi

- Mengatur visualisasi step-by-step
- Menyinkronkan data nama dan nilai saat proses sorting
- Menyimpan setiap langkah proses algoritma

Cara Mengatasi

- Menggunakan struktur data list untuk menyimpan langkah
- Melakukan cloning array pada setiap proses
- Membuat delay untuk visualisasi agar mudah dipahami

Pembagian Tugas Kelompok

- UI / Tampilan: [MAHMUDI DHANI PRATA]
- Implementasi Algoritma: [MAHMUDI DHANI PRATAMA]
- Pengujian & Debugging: [FARHAN DWI RAMADHAN]
- Dokumentasi & Laporan: [FARHAN DWI RAMADHAN]

Kesimpulan akhir:

Pemahaman algoritma menjadi lebih mudah dengan visualisasi. Pemilihan algoritma harus disesuaikan dengan kondisi data dan kebutuhan aplikasi.